

Data Structures & Algorithms

A Complete Study Guide
Big-O Notation . Python Code . Worked Intuition

What's Inside
1 Big-O Notation: how to think about cost
2 Data Structures: from arrays to union-find
3 Searching: linear, binary, jump
4 Sorting: bubble, merge, quick
5 Graph Algorithms: BFS, DFS, Dijkstra
6 Dynamic Programming
7 Patterns: two pointers, windows, backtracking, greedy, topo sort
8 Cheat Sheets

How to use this guide: read each concept's intuition first, then study the code, then cover the code and try to rewrite it from memory. The complexity tables are there to check your understanding, not to memorize blindly. If you can explain why a number is what it is, you actually know it.

1 Big-O Notation: How to Think About Cost

Before we touch a single data structure, you need a way to talk about how *expensive* code is. Imagine two programmers each write a function that finds a name in a list. One is twice as fast on your laptop. Does that make it the better algorithm? Not necessarily, because that gap might vanish, or reverse, on a list a thousand times larger. Big-O exists to strip away the things that depend on your machine and ask the only question that matters at scale: **as the input grows, how fast does the work grow?**

THE MENTAL MODEL

Don't picture a stopwatch. Picture a **graph**. The x-axis is the size of your input (n), the y-axis is the number of steps. Big-O is the *shape* of that curve. A flat line, a gentle curve, a steep wall: those shapes, not the exact numbers, are what we name.

We deliberately ignore constants and small terms. If one function does $2n + 5$ steps and another does n steps, we call them both $O(n)$. That feels wrong at first, but here's the reasoning: when n is a million, the difference between n and $2n$ is a constant multiplier your CPU shrugs off, while the difference between n and n^2 is the difference between a second and two weeks. Big-O keeps your attention on the gap that actually hurts.

The Five Rules You Apply Every Time

Rule	Example	Why
Drop constants	$O(2n)$ becomes $O(n)$	A fixed multiplier doesn't change the shape of the curve.
Drop lower-order terms	$O(n^2 + n)$ becomes $O(n^2)$	At large n the biggest term dwarfs everything below it.
Separate inputs, separate letters	$O(a + b)$, not $O(n)$	Looping over two different arrays is not the same as looping twice over one.
Nested loops multiply	loop in a loop = $O(n^2)$	For each of n outer steps you do n inner steps: n times n .
Sequential steps add	$O(n)$ then $O(m)$ = $O(n+m)$	Work done one after another adds; the dominant term usually wins.

Ranking the Common Curves

FASTEST FIRST

Name	Notation	$n = 10$	$n = 1000$	Feel
Constant	$O(1)$	1	1	Instant
Logarithmic	$O(\log n)$	~3	~10	Excellent
Linear	$O(n)$	10	1,000	Fair

Name	Notation	n = 10	n = 1000	Feel
Linearithmic	$O(n \log n)$	~33	~10,000	Good for sorts
Quadratic	$O(n^2)$	100	1,000,000	Slow
Cubic	$O(n^3)$	1,000	10^9	Very slow
Exponential	$O(2^n)$	1,024	~ 10^{301}	Unusable
Factorial	$O(n!)$	3.6 million	astronomical	Hopeless

* amortized average. a = inverse Ackermann, effectively constant. Notice how $O(\log n)$ barely moves as n explodes, while $O(n^2)$ and worse become unusable almost immediately.

$O(1)$ — Constant Time

The work never changes, no matter how big the input gets. Looking up a value by its array index is the classic case: the computer does one arithmetic calculation to jump straight to that memory location, whether the array holds ten items or ten million.

PICTURE IT

Asking a librarian for the book at shelf 4, row 2, position 7. They walk straight to it. The size of the library is irrelevant to that one lookup.

PYTHON

```
# Every line below is O(1): the work is fixed
arr = [10, 20, 30, 40, 50]

arr[2]           # jump straight to index 2
arr.append(99)   # add to the end (amortized O(1))
arr.pop()        # remove from the end

phone = {"Alice": "555-1234"}
phone["Alice"]   # hash lookup, size doesn't matter

x = 3 * 5 + 2     # arithmetic is always O(1)
```

$O(\log n)$ — Logarithmic Time

Each step throws away half of what's left. That halving is the signature of a logarithm. The magic is how slowly it grows: doubling the input adds just ONE extra step. A billion items takes only about 30 steps. Whenever you see 'cut the problem in half,' think $\log n$.

PICTURE IT

The number-guessing game: I pick a number from 1 to 1000, you guess, I say higher or lower. Guessing the midpoint each time, you always win in about 10 tries because each guess halves the range. That's $\log_2(1000)$.

PYTHON

```
# Binary search - the textbook  $O(\log n)$ 
# REQUIRES a sorted array
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2      # look at the middle
        if arr[mid] == target:
            return mid           # found it
        elif arr[mid] < target:
            lo = mid + 1         # answer must be on the right
        else:
            hi = mid - 1         # answer must be on the left
    return -1

# 1,000,000 items -> at most ~20 comparisons.
# Each loop deletes half the remaining candidates.
```

$O(n)$ — Linear Time

You touch each element a constant number of times, so the work tracks the input one-for-one. Double the data, double the time. This is the floor for any problem where you genuinely have to look at everything at least once, like finding the maximum of an unsorted list.

PICTURE IT

Reading every name on a guest list to check if your friend is invited. No shortcut exists if the list is in random order. You read all n names in the worst case.

PYTHON

```
# Linear search - works on UNSORTED data
def linear_search(arr, target):
    for i, val in enumerate(arr):    # one pass, touch each once
        if val == target:
            return i
    return -1

# Find the largest value
def find_max(arr):
    biggest = arr[0]
    for val in arr:                  # must inspect every element
        if val > biggest:
            biggest = val
    return biggest

# Both grow in a straight line with input size.
```

$O(n \log n)$ — Linearithmic Time

This is what you get when you do an $O(\log n)$ amount of dividing and an $O(n)$ amount of work at each level. It's the speed limit for sorting by comparison: you provably cannot sort faster than this using comparisons alone. When you see a fast sort, this is almost always its cost.

PICTURE IT

Sorting a deck of cards by repeatedly splitting it into halves ($\log n$ splits), then merging the sorted halves back together, where each full merge pass touches all n cards.

PYTHON

```
# Merge sort - divide (log n levels) and merge (n per level)
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])    # split until single items
    right = merge_sort(arr[mid:])
    return merge(left, right)       # then stitch back, sorted

def merge(left, right):
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

# log n levels of splitting x n work per level = n log n
```

$O(n^2)$ — Quadratic Time

Two nested loops over the same data. For each of the n items you do n more units of work, so the total is n times n . This is fine when n is small (a few hundred), but it gets painful fast. Spotting a hidden quadratic and replacing it with a hash map is one of the most common ways to speed up real code.

PICTURE IT

Everyone at a party shaking hands with everyone else. With n guests, that's roughly n times n handshakes. Twice the guests means four times the handshakes.

PYTHON

```
# Quadratic: a loop inside a loop over the same data
def has_duplicate_slow(nums):
    for i in range(len(nums)):          # n iterations
        for j in range(i + 1, len(nums)): # n more each time
            if nums[i] == nums[j]:
                return True
    return False
# n=1000 -> about 500,000 comparisons

# The same task in O(n) using a set:
def has_duplicate_fast(nums):
    seen = set()
    for n in nums:          # one pass
        if n in seen:      # O(1) lookup replaces the inner loop
            return True
        seen.add(n)
    return False
```

$O(2^n)$ — Exponential Time

Every added element doubles the work. This shows up in naive recursion that re-solves the same subproblems over and over, and in problems that explore every possible combination. It becomes impossible to run past about $n=30$. When you hit one of these, the fix is almost always memoization (caching) or dynamic programming.

PICTURE IT

A rumor that doubles its listeners every hour. After n hours, 2^n people know. The growth is fine for a while, then suddenly overwhelming.

PYTHON

```
# Naive Fibonacci re-computes the same values endlessly
def fib_slow(n):
    if n <= 1:
        return n
    return fib_slow(n-1) + fib_slow(n-2) # branches double!
# fib_slow(50) makes ~10^15 calls. Effectively never finishes.

# Cache results -> collapses to O(n)
from functools import lru_cache
@lru_cache(maxsize=None)
def fib_fast(n):
    if n <= 1:
        return n
    return fib_fast(n-1) + fib_fast(n-2) # each n computed once
# fib_fast(50) returns instantly.
```

2 Data Structures

A data structure is just a decision about how to lay your data out so that the operations you care about are cheap. There's no universally best structure, only the best one for what you're about to do. Need instant lookup by a key? A hash map. Need to always grab the smallest item? A heap. Half of getting good at algorithms is recognizing, from the question being asked, which structure quietly makes it easy. Keep that lens as you read each one below.

The Master Reference

MEMORIZE THE SHAPE

Structure	Access	Search	Insert	Delete	Space
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Hash Map / Set	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$
Stack / Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)^+$	$O(1)^+$	$O(n)$
Binary Tree	$O(h)$	$O(h)$	$O(h)$	$O(h)$	$O(n)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Heap	$O(1)$ top	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Trie	$O(m)$	$O(m)$	$O(m)$	$O(m)$	$O(nm)$
Union-Find	$O(a)$	$O(a)$	$O(a)$	$O(a)$	$O(n)$

* amortized average | + $O(1)$ only when you already hold the node | h = height of tree | m = length of the key | a = inverse Ackermann (treat as constant)

Array (Python list)

O(1) access

An array stores its elements back-to-back in one continuous block of memory. That single fact explains all of its behavior. Because the block is continuous, the computer can compute the exact address of element i instantly, which is why access is $O(1)$ and arrays feel so fast in loops. But that same layout is why inserting at the front is slow: there's no gap, so every later element has to physically shift over to make room, which is $O(n)$. Python's list is a *dynamic* array, meaning it quietly grabs a bigger block and copies things over when it runs out of room. Appending is therefore $O(1)$ on average.

WHEN TO REACH FOR IT

Reach for an array when you mostly read by position or append to the end, and when you want your data packed tightly for fast scanning.

PYTHON

```
arr = [10, 20, 30, 40, 50]

arr[2]          # O(1)  jump straight to position 2
arr.append(60)  # O(1)* add to the end
arr.pop()       # O(1)  remove from the end
arr.insert(0, 5) # O(n)  everything must shift right
arr.pop(0)      # O(n)  everything must shift left
30 in arr       # O(n)  must scan to check membership

# A clean O(n) idiom: the sliding window.
# Keep a running total instead of re-summing each time.
def max_window_sum(arr, k):
    total = sum(arr[:k])      # first window
    best = total
    for i in range(k, len(arr)):
        total += arr[i] - arr[i-k] # add new, drop old: O(1)
        best = max(best, total)
    return best
```

Hash Map & Hash Set

O(1) average

A hash map is the workhorse of efficient code. It stores key-value pairs and finds any key in roughly one step by running the key through a hash function that computes *where* the value lives. You don't search; you calculate the location directly. A hash set is the same machine but stores only keys, answering 'have I seen this before?' instantly. The catch is the word 'average': two keys can hash to the same spot (a collision), and in a pathological case everything piles up and degrades to $O(n)$, but with Python's well-built hashing this is rare enough to ignore in practice.

WHEN TO REACH FOR IT

Whenever a brute-force solution loops to *find* or *count* something, ask whether a hash map could replace that inner loop. This single move turns countless $O(n^2)$ solutions into $O(n)$.

PYTHON

```
# Counting frequencies in one pass
freq = {}
for ch in "mississippi":
    freq[ch] = freq.get(ch, 0) + 1    # .get avoids KeyError

# Two Sum: the canonical "hash map kills the inner loop"
def two_sum(nums, target):
    seen = {}                        # value -> index
    for i, n in enumerate(nums):
        need = target - n
        if need in seen:            # O(1), not a loop
            return [seen[need], i]
        seen[n] = i
    return []

# Hash set for instant membership tests
seen = set()
def first_repeat(nums):
    for n in nums:
        if n in seen:                # O(1)
            return n
        seen.add(n)
    return None
```

Stack (Last In, First Out)

O(1) push/pop

A stack only lets you touch the top. You push items on and pop them off the same end, so the last thing in is the first thing out. That restriction sounds limiting but it's exactly what you want for anything with nesting or 'undo' behavior, because the most recent thing is always the thing you need to deal with first. A Python list *is* a stack: append is push, pop is pop, both O(1).

WHEN TO REACH FOR IT

Matching brackets, the back button in a browser, undo in an editor, and depth-first search all run on stacks. If a problem involves 'the most recent unmatched thing,' a stack is your tool.

PYTHON

```
stack = []
stack.append(1)    # push
stack.append(2)
stack[-1]          # peek at the top without removing -> 2
stack.pop()        # remove and return the top

# Valid parentheses: a stack remembers what's still open
def is_valid(s):
    stack = []
    match = {'(': ')', '[': ']', '{': '}'
    for ch in s:
        if ch in '([{':
            stack.append(ch)          # remember an opener
        elif not stack or stack[-1] != match[ch]:
            return False              # nothing matches
        else:
            stack.pop()               # closed correctly
    return not stack                  # all openers resolved?
```

Queue & Deque (First In, First Out)

O(1) ends

A queue is the opposite discipline from a stack: the first item in is the first out, like a line at a checkout. This fairness is what makes it the engine of breadth-first search, where you must explore things in the order you discovered them. One crucial Python detail: never use a plain list and `pop(0)` for a queue, because removing from the front of a list is $O(n)$. Use `collections.deque`, which gives you $O(1)$ at *both* ends.

WHEN TO REACH FOR IT

Reach for a deque whenever order of processing matters and you add to one end while removing from the other, above all in BFS and level-by-level traversals.

PYTHON

```
from collections import deque

q = deque()
q.append(1)      # enqueue at the back    O(1)
q.append(2)
q[0]             # peek at the front      O(1)
q.popleft()      # dequeue from front    O(1)  <- the key win

# A deque works at both ends, unlike a plain queue
dq = deque()
dq.appendleft(0) # O(1)
dq.append(9)     # O(1)

# BFS: process nodes in the order discovered
def bfs(graph, start):
    seen = {start}
    q = deque([start])
    while q:
        node = q.popleft()
        for nxt in graph[node]:
            if nxt not in seen:
                seen.add(nxt)
                q.append(nxt)
```

Linked List

O(1) splice

A linked list scatters its nodes anywhere in memory and threads them together with pointers, each node holding a value and the address of the next. Because nothing is contiguous, there's no formula for 'element i', so reaching a position means walking the chain: $O(n)$. The payoff is surgical insertion and deletion. Once you're standing at a node, you splice in or remove a neighbor by rewiring a pointer or two, no shifting required. The single most important trick to internalize is the **slow and fast pointer**: advance one pointer one step and another two steps to find the middle or detect a loop in a single pass.

WHEN TO REACH FOR IT

Linked lists shine when you insert and delete constantly but rarely need random access. In interviews they're mostly a vehicle for pointer-manipulation puzzles.

PYTHON

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

# Reverse a list by flipping each pointer as you walk
def reverse(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next    # save where we were going
        curr.next = prev  # flip the arrow backwards
        prev = curr        # shuffle both pointers forward
        curr = nxt
    return prev           # prev ends up as the new head

# Find the middle in ONE pass with slow/fast pointers
def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next    # +1 each loop
        fast = fast.next.next # +2 each loop
    return slow              # fast hits the end at 2x speed
```

Binary Tree & Binary Search Tree

O(h)

A binary tree is nodes branching into at most two children. A *binary search tree* adds one powerful rule: everything in a node's left subtree is smaller, everything on the right is larger. That ordering means searching is just repeated halving, like binary search on an array, giving $O(\log n)$ when the tree is balanced. The danger is an unbalanced tree: insert sorted data and it degenerates into a glorified linked list with $O(n)$ operations. The three depth-first traversals are worth memorizing cold, because each has a job. In-order on a BST spits out sorted values; pre-order is for copying or serializing; post-order is for deleting or evaluating from the leaves up.

WHEN TO REACH FOR IT

Trees model anything hierarchical: file systems, decision processes, parsed expressions. A BST specifically gives you sorted data with fast inserts.

PYTHON

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = self.right = None

# In-order (Left, Root, Right) -> sorted output for a BST
def inorder(root):
    if not root: return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# Pre-order (Root, Left, Right) -> good for copying a tree
def preorder(root):
    if not root: return []
    return [root.val] + preorder(root.left) + preorder(root.right)

# How tall is the tree? This drives every O(h) above.
def height(root):
    if not root: return 0
    return 1 + max(height(root.left), height(root.right))

# Is this actually a valid BST? Track the allowed range.
def is_bst(node, lo=float('-inf'), hi=float('inf')):
    if not node: return True
    if not (lo < node.val < hi): return False
    return (is_bst(node.left, lo, node.val) and
            is_bst(node.right, node.val, hi))
```

Heap (Priority Queue)

$O(\log n)$

A heap is a tree that keeps only one promise: the smallest element is always at the root, ready to grab in $O(1)$. It does NOT keep everything fully sorted, which is the trick to its speed; maintaining just that one weak rule after each change costs only $O(\log n)$. Python's `heapq` is a min-heap. There's no max-heap, so the universal hack is to push negated values and negate again on the way out. Whenever a problem says 'k largest', 'k smallest', 'next closest', or 'always process the cheapest option,' a heap is the answer.

WHEN TO REACH FOR IT

Use a heap for streaming top-k problems, for Dijkstra's shortest path, and for any scheduler that always serves the highest-priority job next.

PYTHON

```
import heapq

h = []
heapq.heappush(h, 5)    # O(log n)
heapq.heappush(h, 1)
heapq.heappush(h, 3)
h[0]                    # peek the minimum, O(1) -> 1
heapq.heappop(h)        # remove the minimum, O(log n)

# Turn a whole list into a heap in O(n), faster than n pushes
nums = [9, 4, 7, 1, 2]
heapq.heapify(nums)

# Max-heap trick: store negatives
maxh = []
heapq.heappush(maxh, -10)
biggest = -heapq.heappop(maxh)    # 10

# k largest elements, O(n log k)
def k_largest(nums, k):
    return heapq.nlargest(k, nums)
```

Graph

$O(V + E)$ traversal

A graph is the most general structure here: a set of nodes (vertices, V) connected by edges (E). It can model almost anything with relationships, from road networks to social follows to task dependencies. The standard way to store one is an *adjacency list*, a dictionary mapping each node to its neighbors, which uses only $O(V + E)$ space and is perfect for the sparse graphs you usually meet. The reason nearly every graph problem reduces to BFS or DFS is that 'visit everything reachable' is the foundation you build cycle detection, shortest paths, and connectivity on top of.

WHEN TO REACH FOR IT

Once you can build an adjacency list and run a traversal over it, a huge category of problems becomes approachable. Get fluent at this setup.

PYTHON

```
from collections import defaultdict

# Build an undirected graph as an adjacency list
graph = defaultdict(list)
edges = [(0,1), (0,2), (1,3), (2,4)]
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)    # drop this line for a directed graph

# Count separate "islands" of connected nodes
def count_components(n, edges):
    g = defaultdict(list)
    for u, v in edges:
        g[u].append(v); g[v].append(u)
    seen = set()
    def explore(node):      # depth-first flood
        seen.add(node)
        for nb in g[node]:
            if nb not in seen:
                explore(nb)
    groups = 0
    for node in range(n):
        if node not in seen:
            explore(node)
            groups += 1      # found a brand-new island
    return groups
```

Trie (Prefix Tree)

$O(m)$

A trie stores words by their letters, one character per edge, so words sharing a prefix share a path. The beautiful consequence is that the cost of looking something up depends only on the length of the word, not on how many words you've stored. A million words don't slow down a five-letter lookup. This is what powers autocomplete and spell-check: ask 'what completions start with these letters?' and you just walk down to that point and read off everything below.

WHEN TO REACH FOR IT

Reach for a trie whenever the problem is about prefixes, autocomplete, or searching for many words at once, like finding dictionary words hidden in a grid.

PYTHON

```
class TrieNode:
    def __init__(self):
        self.kids = {}          # letter -> next TrieNode
        self.end = False       # does a word finish here?

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):      # O(m)
        node = self.root
        for ch in word:
            node = node.kids.setdefault(ch, TrieNode())
        node.end = True

    def search(self, word):      # O(m), must end on a word
        node = self._walk(word)
        return node is not None and node.end

    def starts_with(self, prefix): # O(m), any word with prefix
        return self._walk(prefix) is not None

    def _walk(self, s):
        node = self.root
        for ch in s:
            if ch not in node.kids:
                return None
            node = node.kids[ch]
        return node
```

Union-Find (Disjoint Set Union)

$O(a) \sim O(1)$

Union-Find answers one question superbly well: are these two things in the same group? Each element points toward a representative for its group; to test two elements you just check whether they lead to the same representative. Two optimizations make it almost free. *Path compression* flattens the chain every time you look something up, and *union by rank* always attaches the smaller tree under the larger. Together they push the cost down to the inverse Ackermann function, which is below 5 for any input you'll ever see, so we just call it constant.

WHEN TO REACH FOR IT

This is the go-to for connectivity that changes over time, for detecting cycles as you add edges, and as the core of Kruskal's minimum-spanning-tree algorithm.

PYTHON

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))    # each node starts alone
        self.rank = [0] * n

    def find(self, x):
        # who represents x's group?
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])    # compress
        return self.parent[x]

    def union(self, x, y):
        # merge two groups
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False    # already together
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx    # attach smaller under larger
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True

# Detecting a cycle: if union fails, both were already linked
def has_cycle(n, edges):
    uf = UnionFind(n)
    return any(not uf.union(u, v) for u, v in edges)
```

3 Searching: Finding a Needle

Searching boils down to one question with a huge consequence: **is your data sorted?** If it isn't, you have no choice but to look at things one by one. If it is, you can be dramatically cleverer, throwing away half the possibilities with every comparison. That gap, between checking everything and halving repeatedly, is the difference between $O(n)$ and $O(\log n)$, and it's why keeping data sorted is so often worth the trouble.

Linear Search

$O(n)$

The honest, unavoidable approach: start at the beginning and check each element until you find what you want or run out. It's the only thing that works on unsorted data, and it's perfectly fine for small lists. There's nothing clever here, and that's the point. When the data has no structure to exploit, this is the best you can do.

PYTHON

```
def linear_search(arr, target):
    for i, val in enumerate(arr): # check each in turn
        if val == target:
            return i              # found at position i
    return -1                     # walked off the end

# Works on anything, sorted or not, any data type
names = ["Alice", "Bob", "Charlie"]
linear_search(names, "Bob")      # -> 1
```

Binary Search

$O(\log n)$

The reward for keeping data sorted. Look at the middle element; if it's your target you're done; if it's too small the answer must be to the right, so you ignore the entire left half, and vice versa. Every comparison eliminates half of what remains, which is why a million items take only about twenty checks. The most common bug is the loop boundary, so memorize the `lo <= hi` shape. The deeper idea is that you can binary-search not just an array but any 'answer space' where a yes/no condition flips from false to true at one point.

PYTHON

```

def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

# The powerful generalization: search an answer space.
# Find the smallest value where a condition becomes true.
def smallest_valid(lo, hi, ok):
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

```

Jump Search **$O(\sqrt{n})$**

A compromise that's worth understanding even if you rarely reach for it. On a sorted array, jump ahead in fixed strides of about $\sqrt{n}$ until you overshoot the target, then walk backward through that one block with a linear scan. It lands between linear and binary search in speed. It mainly matters in settings where jumping far ahead is cheap but jumping backward is expensive, so you want to limit how far back you ever go.

PYTHON

```

import math

def jump_search(arr, target):
    n = len(arr)
    step = int(math.sqrt(n))
    prev = 0
    # leap forward until we pass the target's block
    while prev < n and arr[min(step, n) - 1] < target:
        prev = step
        step += int(math.sqrt(n))
        if prev >= n:
            return -1
    # linear scan within the identified block
    for i in range(prev, min(step, n)):
        if arr[i] == target:
            return i
    return -1

```

4 Sorting: Putting Things in Order

Sorting is worth studying not because you'll write your own (Python's built-in `sorted` is excellent and you should use it), but because the two great sorting algorithms teach the single most important problem-solving pattern in all of computer science: **divide and conquer**. Break a hard problem into smaller copies of itself, solve those, and combine. Merge sort and quick sort are the same idea pointed in opposite directions, and seeing that is the real lesson here.

Side by Side

Algorithm	Best	Average	Worst	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$
DFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$
Dijkstra	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$	$O(V+E)$

Bubble Sort

$O(n^2)$

We start here only because it's the easiest to see. Repeatedly walk through the list swapping any two neighbors that are out of order; after each full pass the largest remaining element has 'bubbled' to its final spot at the end. It's genuinely slow and you'd never use it in practice, but it makes the cost of nested loops concrete: n passes, each doing up to n comparisons, gives n times n work.

PYTHON

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n - i - 1):      # last i are already done
            if arr[j] > arr[j + 1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:                # nothing moved -> sorted
            break                      # gives  $O(n)$  best case
    return arr
```

Merge Sort

$O(n \log n)$

Divide and conquer in its purest form. Split the array in half, sort each half (by splitting it again, all the way down to single elements which are trivially sorted), then *merge* the two sorted halves by repeatedly taking the smaller front element. The splitting creates $\log n$ levels, and merging touches all n elements at each level, so the cost is a rock-solid $n \log n$ in every case, best or worst. It's also **stable**, meaning equal elements keep their original order, which matters when you sort records by one field and want ties to preserve a previous ordering. The price is $O(n)$ extra memory for the merging.

PYTHON

```
def merge_sort(arr):
    if len(arr) <= 1:          # a single item is sorted
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid]) # conquer each half
    right = merge_sort(arr[mid:])
    return merge(left, right)    # combine them, in order

def merge(left, right):
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:    # <= keeps it stable
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:] # tack on the leftovers
```

Quick Sort

$O(n \log n)$ avg

The mirror image of merge sort: it does its work *before* recursing instead of after. Pick a pivot, then partition the array so everything smaller sits left of it and everything larger sits right; the pivot is now in its final place. Recurse on each side. When pivots split things evenly you get $n \log n$, and because it sorts in place with great cache behavior, it's usually the fastest sorter in practice. The trap is a bad pivot: feed it already-sorted data and naive pivot choice produces lopsided splits and $O(n^2)$. The fix is simply to choose the pivot at random.

PYTHON

```
import random

def quicksort(arr, lo, hi):
    if lo < hi:
        p = partition(arr, lo, hi)
        quicksort(arr, lo, p - 1)    # sort the small side
        quicksort(arr, p + 1, hi)    # sort the large side

def partition(arr, lo, hi):
    # random pivot avoids the O(n^2) trap on sorted input
    r = random.randint(lo, hi)
    arr[r], arr[hi] = arr[hi], arr[r]
    pivot = arr[hi]
    i = lo - 1
    for j in range(lo, hi):
        if arr[j] <= pivot:          # smaller goes to the left
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[hi] = arr[hi], arr[i+1] # pivot to its seat
    return i + 1
```

5 Graph Algorithms: Exploring Connections

Almost every graph question is answered by one of two ways of exploring: go *wide* or go *deep*. Breadth-first search fans out level by level using a queue, and because it reaches everything one step away before anything two steps away, it naturally finds the shortest path in an unweighted graph. Depth-first search plunges down one path as far as it can before backing up, using a stack (often just the call stack via recursion). Master these two and add Dijkstra for when edges have weights, and you can handle the overwhelming majority of graph problems.

Breadth-First Search (BFS)

$O(V + E)$

BFS explores in rings: all the nodes one step from the start, then all the nodes two steps away, and so on. A queue enforces this order, since you process nodes in exactly the order you discover them. The headline property falls right out of that: the first time BFS reaches a node, it has done so by the fewest possible edges, so BFS gives you shortest paths in any unweighted graph for free. The same level-by-level behavior is what you want for level-order traversal of a tree.

PYTHON

```
from collections import deque

# Shortest path (fewest edges) in an unweighted graph
def shortest_path(graph, start, goal):
    q = deque([(start, [start])]) # node + path taken to it
    seen = {start}
    while q:
        node, path = q.popleft() # FIFO keeps rings in order
        if node == goal:
            return path # first arrival = shortest
        for nb in graph[node]:
            if nb not in seen:
                seen.add(nb)
                q.append((nb, path + [nb]))
    return None

# Level-order traversal of a tree uses the same engine
def level_order(root):
    if not root: return []
    out, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)): # drain exactly one level
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        out.append(level)
    return out
```


Depth-First Search (DFS)

$O(V + E)$

DFS commits to a path and follows it to a dead end before backtracking to try alternatives. Recursion expresses this most naturally, where the call stack quietly remembers where to back up to, though you can also use an explicit stack to sidestep Python's recursion limit on deep graphs. DFS is the right tool when you care about reachability, cycles, or exploring every path rather than the shortest one. On a grid it becomes 'flood fill', the trick behind counting islands or filling connected regions.

PYTHON

```
from collections import defaultdict

# Recursive DFS: the call stack tracks where to backtrack
def dfs(graph, node, seen=None):
    if seen is None: seen = set()
    seen.add(node)
    for nb in graph[node]:
        if nb not in seen:
            dfs(graph, nb, seen)
    return seen

# Flood fill: count islands of '1' in a grid
def num_islands(grid):
    if not grid: return 0
    count = 0
    def sink(r, c):
        if (r < 0 or r >= len(grid) or
            c < 0 or c >= len(grid[0]) or grid[r][c] != '1'):
            return
        grid[r][c] = '0'          # mark visited by sinking it
        sink(r+1, c); sink(r-1, c)
        sink(r, c+1); sink(r, c-1)
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == '1':
                sink(r, c)
                count += 1          # one whole island consumed
    return count
```

Dijkstra's Algorithm

$O(E \log V)$

When edges have weights (distances, costs, times), BFS's 'fewest edges' is no longer 'cheapest path', so we need Dijkstra. The idea is greedy: keep a min-heap of nodes ordered by their best-known distance from the start, and always expand the cheapest one next. The first time you pop a node, you've found its true shortest distance, because anything still in the heap costs at least as much. The one firm rule: all weights must be non-negative. A negative edge could make a longer route secretly cheaper, breaking the greedy guarantee, and for that you'd need Bellman-Ford instead.

PYTHON

```
import heapq

def dijkstra(graph, start):
    # graph: node -> list of (neighbor, weight)
    dist = {start: 0}
    heap = [(0, start)]          # (distance so far, node)
    while heap:
        d, node = heapq.heappop(heap) # always the cheapest
        if d > dist.get(node, float('inf')):
            continue              # a better path already found
        for nb, w in graph[node]:
            nd = d + w
            if nd < dist.get(nb, float('inf')):
                dist[nb] = nd      # relax: found a shorter way
                heapq.heappush(heap, (nd, nb))
    return dist

graph = {0: [(1,4),(2,1)], 1: [(3,1)], 2: [(1,2),(3,5)], 3: []}
dijkstra(graph, 0) # {0:0, 1:3, 2:1, 3:4}
```

6 Dynamic Programming

Dynamic programming intimidates people, but the core idea is almost embarrassingly simple: **don't solve the same subproblem twice**. That's it. The reason naive recursion is sometimes catastrophically slow is that it re-derives identical answers over and over; DP just remembers them. A problem is a candidate for DP when it has two features. First, *optimal substructure*: the best solution is built from the best solutions to smaller pieces. Second, *overlapping subproblems*: those smaller pieces keep recurring. When both hold, you have a choice of two styles, and they're really the same idea from opposite directions.

THE TWO STYLES

Top-down (memoization): write the natural recursion, then cache each answer so repeats are free. Easiest to reach for. **Bottom-up (tabulation)**: start from the base cases and fill a table upward until you reach the answer. Uses no recursion and is often more memory-efficient. Pick whichever makes the problem clearer to you.

Memoization: cache the recursion

Top-down

Start with the obvious recursive solution, the one that's correct but slow, then bolt on a cache so each distinct input is computed only once. In Python the cleanest way is the `@lru_cache` decorator, which does the bookkeeping for you. The Fibonacci example below is the canonical demonstration: the naive version is exponential because it recomputes the same values endlessly, and a single line of caching collapses it to linear.

PYTHON

```
from functools import lru_cache

# One decorator turns  $O(2^n)$  into  $O(n)$ 
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)  # each n is now solved once

# The same idea written by hand, for clarity
def fib_manual(n, memo={}):
    if n in memo:
        return memo[n]  # already solved -> reuse
    if n <= 1:
        return n
    memo[n] = fib_manual(n-1, memo) + fib_manual(n-2, memo)
    return memo[n]
```

Tabulation: fill a table

Bottom-up

Instead of recursing down, you start at the base cases and build upward, each entry computed from entries you've already filled. Coin change is a perfect teaching example: to make an amount with the fewest coins, you

reason that the answer is one more coin than the best way to make (the amount minus some coin value), so you try each coin and keep the minimum. By filling amounts 0, 1, 2, up to your target in order, every value you need is already sitting in the table when you reach it.

PYTHON

```
# Fewest coins to make 'amount' from given denominations
def coin_change(coins, amount):
    INF = float('inf')
    dp = [INF] * (amount + 1)
    dp[0] = 0 # zero coins make amount 0
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                # one c-coin plus the best way to make (a - c)
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1

coin_change([1, 5, 11], 15) # 3 -> 5 + 5 + 5
```

A 2-D table: Longest Common Subsequence

$O(m \times n)$

Many DP problems compare two sequences and need a grid rather than a line. LCS asks for the longest subsequence appearing in both strings (letters in order but not necessarily adjacent). Build a table where cell $[i][j]$ is the answer for the first i letters of one string and the first j of the other. The rule is intuitive: if the current letters match, you extend the diagonal answer by one; if they don't, you carry over the better of dropping a letter from either string. This grid pattern reappears in edit distance, sequence alignment, and more.

PYTHON

```
def lcs(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1 # letters match
            else:
                dp[i][j] = max(dp[i-1][j], # drop from a
                               dp[i][j-1]) # or drop from b
    return dp[m][n]

lcs("ABCBDAB", "BDCAB") # 4, e.g. "BCAB"
```

7 Patterns That Solve Whole Families

Beyond named algorithms there are recurring *shapes* of solution. Recognizing the shape is often the whole battle: once you see that a problem is 'really' a sliding window or a backtracking search, the code almost writes itself. These five patterns cover an enormous fraction of interview and real-world problems, so learn to spot the cues that summon each one.

Two Pointers

$O(n)$

Keep two indices and move them intelligently instead of using a nested loop. On a sorted array looking for a pair that sums to a target, start one pointer at each end: if the sum is too big, move the right pointer in to shrink it; too small, move the left pointer out to grow it. Each step rules out a possibility for good, so you finish in one pass instead of n -squared. The cue is a sorted array, or any time you're converging on something from both ends.

PYTHON

```
# Pair that sums to target, in a SORTED array
def two_sum_sorted(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        s = arr[lo] + arr[hi]
        if s == target:
            return [lo, hi]
        elif s < target:
            lo += 1          # need a bigger sum
        else:
            hi -= 1         # need a smaller sum
    return []

# Container with most water: same converging idea
def max_area(h):
    lo, hi, best = 0, len(h) - 1, 0
    while lo < hi:
        best = max(best, min(h[lo], h[hi]) * (hi - lo))
        if h[lo] < h[hi]: lo += 1    # move the limiting wall
        else: hi -= 1
    return best
```

Sliding Window

$O(n)$

When a problem asks about a contiguous run, a substring or subarray, you rarely need to re-examine the whole run each time. Maintain a 'window' and update it incrementally as it slides: add the new element entering on the right, remove the one leaving on the left. For fixed-size windows you keep a running sum; for variable windows you expand the right edge greedily and shrink the left edge whenever a constraint breaks. Either way you turn an $O(n^2)$ re-scan into a single $O(n)$ pass.

PYTHON

```
# Variable window: longest substring with no repeated char
def longest_unique(s):
    last = {}          # char -> most recent index
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1    # jump the left edge past it
        last[ch] = right
        best = max(best, right - left + 1)
    return best

longest_unique("abcabcbb")    # 3  ("abc")

# Fixed window: max sum of k consecutive elements
def max_sum(arr, k):
    total = sum(arr[:k]); best = total
    for i in range(k, len(arr)):
        total += arr[i] - arr[i-k] # slide by one in O(1)
        best = max(best, total)
    return best
```

Backtracking

exponential, pruned

Backtracking is systematic trial and error: build a candidate solution one choice at a time, and the moment a partial choice can't possibly lead anywhere valid, abandon it and back up. The skeleton is always the same three beats: *choose* an option, *recurse* to explore the consequences, then *un-choose* to restore state and try the next option. It generates all subsets, all permutations, all combinations, and solves constraint puzzles like N-Queens and Sudoku. The pruning is what keeps it from being hopeless.

PYTHON

```
# Every subset of nums
def subsets(nums):
    out = []
    def backtrack(start, path):
        out.append(path[:]) # record this partial
        for i in range(start, len(nums)):
            path.append(nums[i]) # choose
            backtrack(i + 1, path) # recurse
            path.pop() # un-choose
    backtrack(0, [])
    return out

# Every permutation of nums
def permutations(nums):
    out = []
    def backtrack(path, used):
        if len(path) == len(nums):
            out.append(path[:]); return
        for i in range(len(nums)):
            if used[i]: continue # skip what's in use
            used[i] = True; path.append(nums[i])
            backtrack(path, used)
            used[i] = False; path.pop() # restore and move on
    backtrack([], [False] * len(nums))
    return out
```

Greedy

often $O(n \log n)$

A greedy algorithm grabs the choice that looks best right now and never reconsiders. That sounds reckless, and for many problems it is wrong, but for a special class it's both correct and beautifully simple. The classic is interval scheduling: to fit the most non-overlapping meetings in a day, repeatedly take the meeting that *ends* earliest, because finishing soonest leaves the most room for the rest. The hard part of greedy isn't the code, it's convincing yourself the local choice is also globally optimal; when in doubt, reach for DP instead.

PYTHON

```
# Maximum non-overlapping intervals: pick earliest finishers
def max_meetings(intervals):
    intervals.sort(key=lambda x: x[1])    # sort by END time
    count, end = 0, float('-inf')
    for start, finish in intervals:
        if start >= end:                  # doesn't overlap
            count += 1
            end = finish                  # commit to this one
    return count

# Merge overlapping intervals
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    out = [intervals[0]]
    for s, e in intervals[1:]:
        if s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)    # extend
        else:
            out.append([s, e])                  # start fresh
    return out
```

Topological Sort

$O(V + E)$

When tasks depend on other tasks, you need an order that respects every dependency, like choosing the sequence to take courses with prerequisites. Topological sort produces such an order for a directed acyclic graph. Kahn's algorithm is the cleanest approach: count how many prerequisites each node has (its in-degree), start with the ones that have none, and each time you place a node, decrement its dependents' counts, adding any that hit zero. If you can't place everything, the graph had a cycle, which means the dependencies were impossible to satisfy.

PYTHON

```
from collections import deque

# Order tasks so every prerequisite comes first
def topo_sort(n, prereqs):
    graph = [[] for _ in range(n)]
    indegree = [0] * n
    for course, need in prereqs:
        graph[need].append(course) # 'need' must come first
        indegree[course] += 1

    # begin with everything that has no prerequisites
    q = deque(i for i in range(n) if indegree[i] == 0)
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nxt in graph[node]:
            indegree[nxt] -= 1 # one prerequisite cleared
            if indegree[nxt] == 0:
                q.append(nxt) # now unblocked

    # placed everything? then no cycle existed
    return order if len(order) == n else []
```

8 Cheat Sheets

These condense everything above into quick-glance references. Use them to test yourself: cover the right-hand columns and see if you can reproduce the complexities and the reasons.

Big-O Growth

Name	Notation	n = 10	n = 1000	Feel
Constant	$O(1)$	1	1	Instant
Logarithmic	$O(\log n)$	~3	~10	Excellent
Linear	$O(n)$	10	1,000	Fair
Linearithmic	$O(n \log n)$	~33	~10,000	Good for sorts
Quadratic	$O(n^2)$	100	1,000,000	Slow
Cubic	$O(n^3)$	1,000	10^9	Very slow
Exponential	$O(2^n)$	1,024	~ 10^{301}	Unusable
Factorial	$O(n!)$	3.6 million	astronomical	Hopeless

Data Structure Operations

Structure	Access	Search	Insert	Delete	Space
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Hash Map / Set	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$
Stack / Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)^+$	$O(1)^+$	$O(n)$
Binary Tree	$O(h)$	$O(h)$	$O(h)$	$O(h)$	$O(n)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Heap	$O(1)$ top	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Trie	$O(m)$	$O(m)$	$O(m)$	$O(m)$	$O(nm)$
Union-Find	$O(a)$	$O(a)$	$O(a)$	$O(a)$	$O(n)$

Which Pattern, and Why

If the problem...	Reach for	Because
involves a sorted array	Binary search / two pointers	Order lets you skip half the work or converge from both ends.
asks about a contiguous subarray/substring	Sliding window	You can update the window in $O(1)$ instead of re-scanning.
needs fast lookup, counting, grouping	Hash map / set	$O(1)$ average access replaces an inner search loop.
wants the k largest/smallest, or a scheduler	Heap	The extreme element is always ready in $O(1)$.
asks for shortest path, no weights	BFS	Exploring in rings reaches every node by the fewest edges first.
asks for shortest path with weights	Dijkstra	A min-heap always expands the cheapest frontier node.
explores all paths, combos, arrangements	DFS / backtracking	Choose, recurse, un-choose explores the whole space with pruning.
has tasks with dependencies (a DAG)	Topological sort	Produces an order respecting every prerequisite.
merges or detects groups as edges arrive	Union-Find	Near-constant 'same group?' and 'merge groups' operations.
optimizes with reused subproblems	Dynamic programming	Caching subproblem answers removes exponential repetition.
is about prefixes or autocomplete	Trie	Lookup cost depends only on word length, not dictionary size.

Python Tools You'll Use Constantly

Tool	Does	Cost
<code>list.append(x) / list.pop()</code>	Add or remove at the end (use as a stack)	$O(1)^*$
<code>list.pop(0)</code>	Remove from front - AVOID, use deque	$O(n)$
<code>collections.deque</code>	$O(1)$ at both ends (use as a queue)	$O(1)$
<code>heapq.heappush / heappop</code>	Min-heap insert / remove smallest	$O(\log n)$
<code>heapq.heapify(lst)</code>	Turn a list into a heap in place	$O(n)$
<code>heapq.nlargest(k, it)</code>	The k largest items	$O(n \log k)$
<code>sorted(it) / list.sort()</code>	Timsort, stable	$O(n \log n)$
<code>collections.Counter(it)</code>	Frequency dictionary in one pass	$O(n)$

Tool	Does	Cost
<code>collections.defaultdict</code>	Auto-creates missing keys (great for graphs)	$O(1)^*$
<code>set(it) / x in set</code>	Deduplicate; $O(1)$ membership tests	$O(1)^*$
<code>bisect.bisect_left</code>	Binary search on a sorted list	$O(\log n)$
<code>functools.lru_cache</code>	Automatic memoization for DP	$O(1)/\text{call}$

Data Structures & Algorithms Study Guide . Python Edition
Read the intuition, study the code, then rebuild it from memory.